

Tracking agent KPIs and contract changes: data model design

A monthly validation model that decides when an agent moves between salary-based and commission-based contracts, with full history and verified logic.

5 + 1

tables + reporting view

1 row

per agent per month

2

independent implementations agree

THE BUSINESS PROBLEM

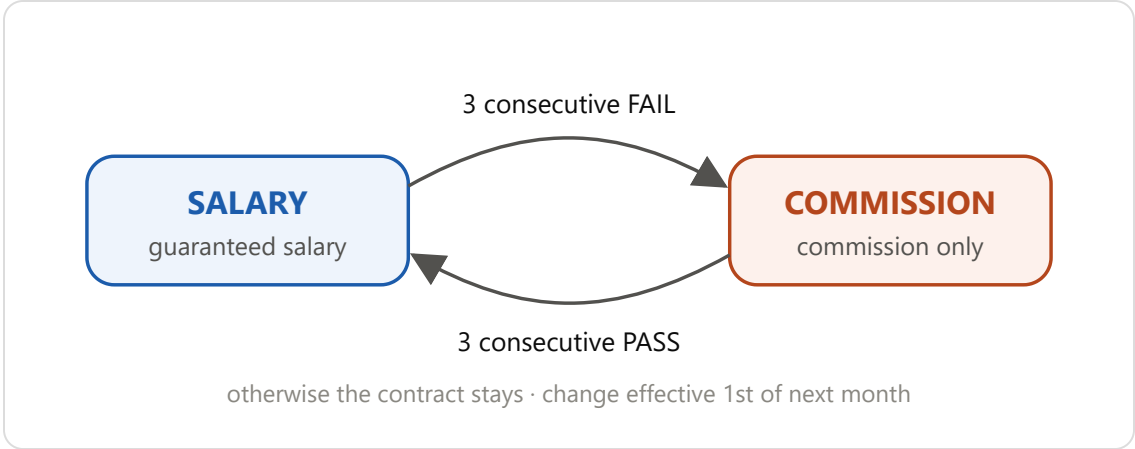
The sales KPI changed: monthly results now decide the agent's contract

The rule from the brief, and the contract life-cycle it creates

Monthly pass

Total premium > 15,000 THB AND new policies > 5

Condition	Result
Fail 3 months in a row	Contract → commission-based
Pass 3 months in a row	Contract → salary-based
Anything else	Contract stays the same



Design need: the tables must record a result **every month**, remember **consecutive streaks**, and keep a **history** of every contract change.

ASSUMPTIONS

Six decisions where the brief was silent

Stated up front so the business can confirm or change them; each one is a single setting in the design

- ❶ **Total premium** = premium of new policies issued in the month; **cancelled policies don't count**.
- ❷ Thresholds are **strict**: exactly 15,000 THB or exactly 5 policies is a **fail**.
- ❸ Every agent employed during the month is evaluated, and **no sales = fail**.
- ❹ A contract change takes effect on the **1st of the following month**.
- ❺ A change is only recorded if the contract **actually changes** (no repeated "to commission").
- ❻ Streaks reset only on the **opposite result**.

Five tables and one view, each with a single job

The core idea: one row per agent per month, carrying the running streaks

Table	Grain	Purpose
agent	1 / agent	Master data + current contract
policy	1 / policy	Sales transactions the KPI is calculated from
kpi_rule	1 / rule version	Thresholds as data, versioned by date
agent_monthly_kpi	1 / agent / month	Result, streaks and contract decision
agent_contract_history	1 / period	Audit trail of every contract change
v_agent_kpi_status	view	Who is at risk, and how soon

- 1

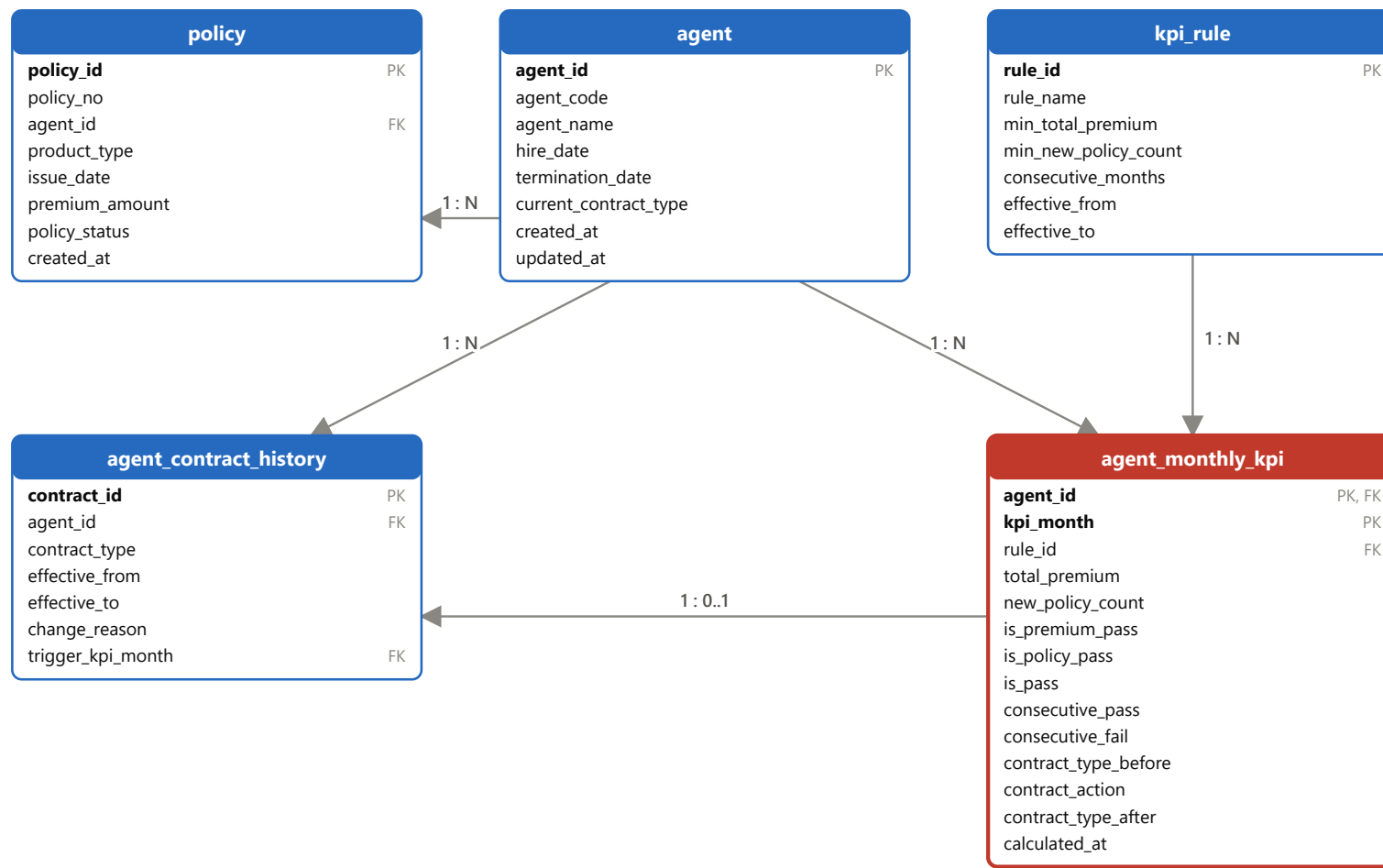
Stored streaks: each month needs only the previous month's row, so there's no re-scan of history.
- 2

Rules are data: the KPI already changed once; the next change is an INSERT, not a code release.
- 3

History table: answers "what contract did this agent have in March, and why?"

ER diagram: everything hangs off the agent and the monthly result

Generated from the live database schema · red = core tracking table · 1 : N = one parent, many children



agent_monthly_kpi: one row explains the whole decision

Primary key (agent_id, kpi_month) guarantees exactly one result per agent per month

1 • What was measured

total_premium

new_policy_count

Sum and count of the month's non-cancelled new policies.

2 • Did they pass?

is_premium_pass

is_policy_pass

is_pass

Separate flags show *why* an agent failed, which is useful for coaching.

3 • How long is the streak?

consecutive_pass

consecutive_fail

Previous value + 1, or reset to 0. Shows how close an agent is to a change.

4 • What did we decide?

contract_type_before

contract_action

contract_type_after

NONE / TO_COMMISSION / TO_SALARY, fully explainable from one row.

Why store streaks instead of recalculating? The decision depends on unbroken runs of months; storing the running count makes each month a simple, fast, auditable step.

A four-step job runs once a month, safely

sql/monthly_kpi_job.sql, run with :kpi_month = the first day of the month just closed

1 Evaluate

Sum premium and count policies for every agent employed in the month; compare with the rule in force.

2 Streaks

Pass: pass streak + 1, fail streak = 0. Fail: the opposite.

3 Decide

SALARY with fail streak $\geq 3 \rightarrow$ commission. COMMISSION with pass streak $\geq 3 \rightarrow$ salary.

4 Apply

Close the old contract, open the new one from the 1st of next month, update the agent.

Safeguards

- All 4 steps in **one transaction**: all or nothing.
- A month **cannot run twice** (code check + primary key).
- A month **cannot run before the previous one** (streaks depend on it).

Effective dating

- Contracts use effective_from / effective_to (exclusive).
- The current contract has effective_to = NULL.
- Any past date can be answered: "which contract was in force on X?"

WORKED EXAMPLE

One agent, eight months: two contract changes

Agent A002 · green = pass, red = fail · bottom row = contract in force · → = change decided that month

		Jan	Feb	Mar	Apr	May	Jun	Jul	Aug
A002 Busaba	KPI	✓ PASS 25.0k · 7 pol streak 1	✗ FAIL 9.0k · 3 pol streak 1	✗ FAIL 9.0k · 3 pol streak 2	✗ FAIL 9.0k · 3 pol streak 3	✓ PASS 25.0k · 7 pol streak 1	✓ PASS 25.0k · 7 pol streak 2	✓ PASS 25.0k · 7 pol streak 3	✓ PASS 25.0k · 7 pol streak 4
	Contract	Salary	Salary	Salary	Salary → Commission	Commission	Commission	Commission → Salary	Salary

- 1

Feb–Apr: three fails in a row (9k premium, 3 policies), and the fail streak reaches 3 in April.
- 2

April decision: **TO_COMMISSION**, effective **1 May**.
- 3

May–Jul: three passes while on commission; the pass streak reaches 3 in July.
- 4

July decision: **TO_SALARY**, effective **1 August**.

DEMO RESULTS

Four agents chosen to exercise every rule

Real output of the monthly SQL job, Jan–Aug 2026

		Jan	Feb	Mar	Apr	May	Jun	Jul	Aug
A001	KPI	✓ PASS 25.0k · 7 pol streak 1	✓ PASS 25.0k · 7 pol streak 2	✓ PASS 25.0k · 7 pol streak 3	✓ PASS 25.0k · 7 pol streak 4	✓ PASS 25.0k · 7 pol streak 5	✓ PASS 25.0k · 7 pol streak 6	✓ PASS 25.0k · 7 pol streak 7	✓ PASS 25.0k · 7 pol streak 8
	Contract	Salary	Salary	Salary	Salary	Salary	Salary	Salary	Salary
A002	KPI	✓ PASS 25.0k · 7 pol streak 1	✗ FAIL 9.0k · 3 pol streak 1	✗ FAIL 9.0k · 3 pol streak 2	✗ FAIL 9.0k · 3 pol streak 3	✓ PASS 25.0k · 7 pol streak 1	✓ PASS 25.0k · 7 pol streak 2	✓ PASS 25.0k · 7 pol streak 3	✓ PASS 25.0k · 7 pol streak 4
	Contract	Salary	Salary	Salary	Salary → Commission	Commission	Commission	Commission → Salary	Salary
A003	KPI	✗ FAIL 9.0k · 3 pol streak 1	✗ FAIL 9.0k · 3 pol streak 2	✓ PASS 25.0k · 7 pol streak 1	✗ FAIL 9.0k · 3 pol streak 1	✗ FAIL 9.0k · 3 pol streak 2	✗ FAIL 9.0k · 3 pol streak 3	✓ PASS 25.0k · 7 pol streak 1	✗ FAIL 9.0k · 3 pol streak 1
	Contract	Salary	Salary	Salary	Salary	Salary	Salary → Commission	Commission	Commission
A004	KPI	✗ FAIL 15.0k · 6 pol streak 1	✗ FAIL 30.0k · 5 pol streak 2	✓ PASS 25.0k · 7 pol streak 1	✓ PASS 25.0k · 7 pol streak 2	✓ PASS 25.0k · 7 pol streak 3	✗ FAIL 9.0k · 3 pol streak 1	✗ FAIL 9.0k · 3 pol streak 2	✗ FAIL 9.0k · 3 pol streak 3
	Contract	Commission	Commission	Commission	Commission	Commission → Salary	Salary	Salary	Salary → Commission

A003: a single pass in March resets the fail streak, so no change until Apr–Jun. **A004:** exactly 15,000 THB (Jan) and exactly 5 policies (Feb) both fail.

The tricky cases are handled and tested

Each row is an automated test that runs the real SQL job

Case	Expected behaviour	Status
Premium exactly 15,000 / exactly 5 policies	Fail (the rule says "more than")	✓ tested
Month with no sales	Agent still evaluated, result = fail	✓ tested
Cancelled policy	Excluded from premium and count	✓ tested
Fail, fail, pass, fail, fail	Streak resets; no contract change	✓ tested
Commission agent keeps failing	No repeated change, no duplicate history	✓ tested
Agent hired mid-month	Evaluated from the hiring month	✓ tested (bug found and fixed)
Same month run twice / month skipped	Rejected with a clear error	✓ tested
KPI month not the 1st	Rejected	✓ tested

VERIFICATION

Two independent implementations give identical results

The SQL job vs a separate pure-Python version of the same rules (ReferenceKpiEngine)

Agent-months compared

7,234

+ 32 demo rows · 30 random portfolios

Result

identical

every column, every row

Automated tests

25

rules, safeguards, equivalence

- 1 Random portfolios include **mid-month hires, leavers and cancelled policies**, the cases people forget.
- 2 If two independent versions agree on thousands of cases, both are very likely right.

It caught a real bug: agents hired mid-month were silently skipped in their first month, because the job looked for a contract in force on the 1st. It's now fixed and has a test.

One query answers "who is about to lose their salary contract?"

The view v_agent_kpi_status shows each agent's latest position and months until a change

Example query for the sales manager

```
SELECT agent_code, consecutive_fail,
       months_to_change
FROM   v_agent_kpi_status
WHERE  status = 'AT RISK'
ORDER BY months_to_change;
```

- 1 **AT RISK:** salary agent with ≥ 1 fail in a row; a coaching trigger.
- 2 **RECOVERING:** commission agent on a pass streak.

Demo output after the August run

Agent	Contract	Last month	Result	Pass streak	Fail streak	Months to change	Status
A001	SALARY	2026-08	PASS	8	0	3	STABLE
A002	SALARY	2026-08	PASS	4	0	3	STABLE
A003	COMMISSION	2026-08	FAIL	0	1	3	STABLE
A004	COMMISSION	2026-08	FAIL	0	3	3	STABLE

Built to change: new rules, more agents, more uses

What the design already supports, and what I would add next

- 1 KPI changes again?** Insert a new `kpi_rule` row with an effective date; past months keep their old rule.
- 2 Scale:** 2,000 agents $\times$ 12 months = 24,000 rows a year. That's tiny, and indexed on (agent, month).
- 3 Portable SQL:** runs on SQLite for the demo; the same design works on PostgreSQL, SQL Server or Oracle.
- 4 Early warning:** alert agent and manager at fail streak 2.
- 5 Late cancellations:** a controlled re-run from the affected month forward.
- 6 Integration:** feed contract changes to HR and payroll; add a manager dashboard.

THANK YOU

Questions?

Full answer document: **Answer Case 1B.pdf** · SQL: **sql/** · demo database: **output/agent_kpi_demo.db** ·
run: **python main.py**